



MANUAL TEST GUIDE · MODULE 10 OF 11

Shipment Profitability Functional-Flow Test Walkthroughs

Scenario-driven, hands-on QA guide for the **logistics.profitability** operational layer — charge-line revenue/cost, gross-profit & margin roll-up, multi-currency conversion, the approval-gated adjustment lifecycle, and the append-only audit ledger. This is an **operational** profitability view, not a finance system: Phase 1 runs on estimated cost + inherited revenue.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives you a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

01 What Profitability does & how to read this guide

Profitability is the **operational** answer to "is this shipment making money?" — a read-friendly roll-up over a shipment's economics that *never* becomes an accounting system. The atomic unit is the `logistics.profitability.charge.line` (one revenue/cost pair per charge); the roll-up surface is the `logistics.shipment.profitability` header. Phase 1 runs entirely on **estimated cost + inherited revenue** — no finance sync, no invoices, no vendor bills.

6

COMMANDS TO EXERCISE

3

TEST SCENARIOS

5

CORE MODELS

~27%

DEMO GP MARGIN

The three state vocabularies you will observe

The header carries a **lifecycle stage**, every adjustment a **lifecycle status**, and every header a derived **margin flag**. Watch all three on the Profitability tab.

stage

: estimated → provisional → actual → closed (Phase 1 stays at **estimated**)

adj status:

draft → pending_approval → approved → applied | rejected

margin flag:

healthy | warning | critical | negative (derived from margin_pct vs thresholds)

The two engines behind every number

ROLL-UP (SERVICES/ROLLUP.PY)

- Header stores **no** hand-entered totals (PRF-BR-02)
- Sums the charge lines' **base-currency** amounts
- Gross profit = revenue – effective cost (actual where synced, else estimated)
- Fires from charge-line post-create / post-update / pre-delete hooks

CURRENCY (SERVICES/CURRENCY.PY)

- Resolves revenue → base & cost → base rates (PRF-20)
- Writes `converted_revenue_amount` / `converted_cost_amount` onto the line
- Single-currency lines convert at rate 1 (identity)
- Upserts a `logistics.profitability.currency.line` FX record

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Initialize & roll-up	Inherit charge lines from the shipment's frozen charges, recompute the header, verify GP & margin match the demo figures; RBAC gating	<code>initialize</code> , <code>recompute</code>	§02
B — Multi-currency	A charge bought in INR / sold in USD; the currency line's converted amounts feed the base-currency roll-up	<code>recompute</code>	§03
C — Adjustment lifecycle	submit → approve → apply through <code>foundation.approval</code> ; audit trail + re-rolled margin; reject path	<code>adjustment</code> <code>submit/approve/reject/apply</code>	§04
Guards	Every negative that must be refused — status-order guards, append-only ledger, immutable cost source	all six	§05

Demo fixtures you start from (loaded by `--with-demo=all`)

Header `profitability.demo_header_globex_001` on the Globex shipment `CO-SEA-D-2026-000001` (base **USD**, stage **estimated**). Five charge lines — freight 3200/2400, origin 400/250, destination 300/200, customs 150/100, docs 100 USD / 5000 INR — roll up to **revenue 4150 • est cost 3010.24 • GP 1139.76 • margin 27.46%** (healthy). A pending detention-recovery adjustment (+200 USD) and two audit entries complete the surface.

02 Initialize & roll-up — the profit object comes to life

Goal: stand up a profitability header over the Globex shipment, prove revenue/cost are **inherited not re-keyed** (PRF-BR-03), and confirm the roll-up matches the demo figures. **Role:** Profitability Editor. **Start state:** shipment `CO-SEA-D-2026-000001` with frozen charges.

#	Action	Expected result	✓ Checkpoint	Trace
1	Dispatch <code>logistics.profitability.initialize</code> with payload <code>{"shipment_id": <uuid>}</code> .	A <code>logistics.shipment.profitability</code> header is created (base = USD, stage estimated). One <code>charge.line</code> per frozen shipment charge is inherited: sell → <code>revenue_amount</code> , buy → <code>estimated_cost_amount</code> , source stamped.	Header exists; charge lines populated from the shipment — nothing re-typed.	PRF-01/02 PRF-06
2	Dispatch <code>initialize</code> a second time against the same shipment.	Idempotent — the existing header UUID is returned; no duplicate header or duplicate lines are created.	Same profitability_id ; line count unchanged.	PRF-BR-02
3	Read the freight charge line's computed fields.	<code>profit_amount</code> = 800 (3200 – 2400); <code>margin_pct</code> = 25.00. Computed in base currency, stored.	Per-line profit & margin auto-computed, not entered.	PRF-BR-05
4	Force a roll-up: dispatch <code>logistics.profitability.recompute</code> on the header.	Header totals re-derived from the lines: total_revenue 4150 · total_estimated_cost 3010.24 · gross_profit 1139.76 · margin_pct 27.46.	Roll-up totals equal the sum of the five lines.	PRF-BR-02 PRF-10/11
5	Read the header <code>margin_flag</code> .	27.46% is above the warning threshold (default 15) so the flag is healthy . Gross profit > 0 so the negative branch does not fire.	margin_flag = healthy .	margin cfg
6	Log in as a user without a profitability role and open the shipment's Profitability tab.	The commercially-sensitive surface is gated by <code>ir.rbac</code> (roles <code>profitability_viewer</code> / <code>editor</code> / <code>approver</code>); a non-role user cannot read the header / lines / margins.	Profitability hidden from non-role users.	PRF-BR-17

✓ Scenario A pass criteria

The header materialised from carried-forward charges, per-line profit/margin computed automatically, and the roll-up reconciled to **GP 1139.76 / margin 27.46%** — with the surface visible only to profitability roles.

⚠ Header totals are never authored

You cannot type totals onto the header — they are recomputed from the charge lines on every line create/update/delete (PRF-BR-02). If a total looks wrong, the fix is on a **line**, then let the hook (or `recompute`) re-roll it.

03 Multi-currency — one margin, many currencies

Goal: prove that a charge sold in one currency and bought in another still rolls up to a **single base-currency margin** — freight forwarding is inherently multi-currency (sell USD, buy EUR/INR, roll up in the org base). **Start:** the seeded **documentation** line — revenue 100 USD, estimated cost **5000 INR**, base USD.

```
docs line: revenue 100 USD — rate 1.0 → converted_revenue 100.00 USD
              cost 5000 INR — rate 0.012048 → converted_cost 60.24 USD → profit 39.76 USD
```

#	Action	Expected result	✓ Checkpoint	Trace
1	Open the documentation charge line (<code>DOC-FEE</code>) and read its currencies.	<code>revenue_currency_id</code> = USD, <code>cost_currency_id</code> = INR, <code>base_currency_id</code> = USD — a genuinely cross-currency line.	Revenue & cost currencies differ.	PRF-20
2	Read its <code>converted_revenue_amount</code> / <code>converted_cost_amount</code> .	Revenue converts at rate 1 (already USD) → 100.00; cost converts INR → USD at 0.012048 → 60.24 . These base amounts drive the line's profit compute.	converted_cost = 60.24 USD.	PRF-BR-11
3	Open the matching <code>logistics.profitability.currency.line</code> FX record.	One FX row per cross-currency line: <code>cost_exchange_rate</code> = 0.012048 , <code>exchange_rate_source</code> = <code>res_exchange_rate</code> , rate date, converted amounts, <code>fx_gain_loss_amount</code> = 0 (realised only when actuals sync).	FX record captures rate, source & date.	PRF-20
4	Read the docs line <code>profit_amount</code> .	Profit computes in base currency from the converted amounts: 100.00 – 60.24 = 39.76 USD — <i>not</i> 100 – 5000.	Profit uses converted, not raw, cost.	PRF-BR-05
5	<code>recompute</code> the header and confirm the total.	The roll-up sums converted base amounts (falling back to raw only for single-currency lines), so the docs cost enters as 60.24 — total est cost stays 3010.24 and margin 27.46% .	Converted amount folds into the base total.	PRF-BR-02
6	Inspect the four USD-only lines (freight / origin / destination / customs).	Single-currency lines carry <code>converted_*</code> equal to the raw amounts (rate 1); the roll-up treats them identically.	Identity conversion on same-currency lines.	PRF-20

✓ **Scenario B pass criteria**

The INR cost was resolved to 60.24 USD via the currency engine, stored on both the charge line and its FX record, and folded into a single USD margin — the header never mixes currencies.

⚠ **Rate source drives the number**

The applied rate comes from `profitability.exchange_rate_source` (default `res_exchange_rate` → latest `res.exchange.rate` for from → base). If no rate row exists, the engine falls back to identity (rate 1) — a missing FX master silently understates a cross-currency cost. Confirm the rate row exists before trusting the converted total.

04 Adjustment lifecycle — approval, apply & audit

Goal: override an inherited amount the **auditable** way — never by silently editing a charge line (PRF-BR-03). Drive the seeded detention-recovery adjustment (+200 USD on the freight line) submit → approve → apply, then confirm the header re-rolls and the audit ledger records it. **Roles:** Editor submits; Approver approves.

#	Action	Expected result	✓ Checkpoint	Trace
1	Open the seeded adjustment <code>demo_adjustment_detention</code> — type <code>revenue_increase</code> , amount 200 USD, <code>status = pending_approval</code> , <code>requires_approval = true</code> .	Because <code>profitability.revenue_adjustment_requires_approval</code> is on (default true), the delta sits in pending_approval awaiting sign-off — it has not touched the freight line yet.	Line still reads 3200; adjustment pending.	PRF-06
2	As Approver, dispatch <code>logistics.profitability.adjustment.approve</code> .	Status → approved then immediately applied (approve calls apply). The freight <code>revenue_amount</code> becomes 3200 + 200 = 3400 ; <code>revenue_source</code> flips to <code>manual_adjustment</code> .	Freight revenue now 3400; status applied.	PRF-06
3	Read the adjustment's <code>before_amount</code> / <code>after_amount</code> .	Stamped for audit: before 3200, after 3400.	Before/after captured on the adjustment.	PRF-06
4	Read the header totals again.	Apply recomputes the roll-up: total revenue rises to 4350 , gross profit to 1339.76 , margin re-derived — no manual re-roll needed.	Header re-rolled after apply.	PRF-BR-02
5	Open the <code>logistics.profitability.audit.entry</code> trail for the header.	A new <code>manual_override</code> entry is appended with the before/after JSON, the reason, and a pointer to the adjustment record.	Audit entry written for the apply.	PRF-BR-18
6	Reject path (separate adjustment): raise a new draft, <code>submit</code> it, then dispatch <code>adjustment.reject</code> .	Status → rejected ; no charge-line mutation occurs, and a <code>manual_override</code> audit entry records the rejection reason.	Reject leaves lines untouched but audited.	PRF-06 PRF-BR-18

✓ Scenario C pass criteria

The +200 delta reached the freight line **only** after approval, the header re-rolled automatically to GP 1339.76, and both the apply and the reject wrote immutable audit entries — the charge line was never edited directly.

⚠ Approval is config-gated

When `profitability.revenue_adjustment_requires_approval` is **off**, `submit` auto-approves and applies in one step (no pending state). Verify the config value before asserting where a freshly-submitted adjustment lands.

05 The guards that must hold

Half of testing a commercially-sensitive surface is confirming it **refuses** the wrong move. Each row below trips a real `ValueError` — the message shown is the exact string from the module source.

Adjustment status-order guards

#	Action	Expected refusal (verbatim)	✓
1	Dispatch <code>adjustment.apply</code> on an adjustment that is still pending_approval / draft (not approved).	"only an approved adjustment can be applied"	Blocked
2	Dispatch <code>adjustment.submit</code> on an adjustment whose status is not draft .	"only a draft adjustment can be submitted"	Blocked
3	Dispatch <code>adjustment.approve</code> on an adjustment that is not pending_approval / approved .	"only a pending adjustment can be approved"	Blocked
4	Dispatch <code>adjustment.reject</code> on an adjustment that is not pending_approval .	"only a pending adjustment can be rejected"	Blocked
5	Apply a <code>revenue_increase</code> / <code>cost_increase</code> (etc.) adjustment with no target charge line.	"adjustment_type '...' requires a target charge line"	Blocked

Ledger & immutability guards

#	Action	Expected refusal (verbatim)	✓
6	Attempt to update any <code>logistics.profitability.audit.entry</code> row.	"profitability audit entries are append-only and cannot be updated (PRF-BR-18)"	Blocked
7	Attempt to delete any <code>logistics.profitability.audit.entry</code> row.	"profitability audit entries are append-only and cannot be deleted (PRF-BR-18)"	Blocked
8	Change a charge line's <code>estimated_cost_source</code> to a different value once it is already set.	"estimated_cost_source is immutable once set (PRF-BR-04) — re-estimation must create a new audit entry, not overwrite the source"	Blocked
9	Dispatch <code>logistics.profitability.initialize</code> with an empty payload (no <code>shipment_id</code>).	"logistics.profitability.initialize requires payload.shipment_id"	Blocked

Honest scope — what Phase 1 does NOT do

Actual cost / actual revenue stay 0 (no finance sync); **billing_status**, **cost_status**, **finance_sync_status**, and the **variance** fields are Phase-2 placeholders. **Closure**, **exceptions**, and **master/console cost allocation** are Phase 2. **Leg-wise profitability**, **AI estimates**, and **BI/reports** are Phase 3. Do not test these here — the enum values exist so later phases emit into them, but no Phase-1 command populates them.

✓ Full-module pass criteria

All three scenarios green — initialize + roll-up to GP 1139.76 / 27.46% (A), the INR → USD line folded into one base margin (B), the approval → apply → audit lifecycle with a re-rolled header and a clean reject path (C) — and every guard in §05 confirmed to refuse with its exact message.

Traceability: PRF- / PRF-BR-* IDs are the requirement tags carried in the `logistics.profitability` module source (models & services), mapped to the Shipment Profitability BRD (BRD 6). Cross-module effects (shipment charges, approval cases) are verified in those modules' own test guides.*